ಓವರ್‌ಫಿಟ್ಟಿಂಗ್ ನಿಯಂತ್ರಣ ಮತ್ತು ಪಠ್ಯ ಎನ್‌ಕೋಡಿಂಗ್

Regularisation, Dropout, Batch Normalization, Cross-Validation, IMDB ಸೆಂಟಿಮೆಂಟ್ ವಿಶ್ಲೇಷಣೆ

ಸರಳ ಕನ್ನಡದಲ್ಲಿ, ದಿನನಿತ್ಯದ ಉದಾಹರಣೆಗಳು ಮತ್ತು ಹಂತ-ಹಂತದ ಲೆಕ್ಕಗಳೊಂದಿಗೆ

“Lecture 6: Controlling Overfitting — Techniques and Best Practices. Text Preprocessing and Encoding”
ಉಪನ್ಯಾಸ ಸ್ಲೈಡ್‌ಗಳನ್ನು ಆಧರಿಸಿದ ಕನ್ನಡ ಅಧ್ಯಯನ ಮಾರ್ಗದರ್ಶಿ

ಪರಿವಿಡಿ

೧. ದೊಡ್ಡ ಚಿತ್ರ: ಈ ಉಪನ್ಯಾಸದ ನಕ್ಷೆ

೨. Regularisation — ಮಾದರಿಗೆ “ಲಗಾಮು”

೩. L2 (Ridge) Regularisation

೪. L1 (Lasso) Regularisation ಮತ್ತು L1 vs L2 ಹೋಲಿಕೆ

೫. Elastic Net ಮತ್ತು Keras ನಲ್ಲಿ regularizers

೬. Early Stopping (ಪುನರಾವಲೋಕನ) ಮತ್ತು Dropout

೭. Batch Normalization

೮. Validation ತಂತ್ರಗಳು: Hold-out ಮತ್ತು K-Fold

೯. Stratified, ShuffleSplit, Leave-P-Out, TimeSeriesSplit

೧೦. ಪಠ್ಯವನ್ನು ಸಂಖ್ಯೆಗಳಾಗಿಸುವುದು: IMDB ಸೆಂಟಿಮೆಂಟ್ ವಿಶ್ಲೇಷಣೆ, Tokenizer

೧೧. ವಿಧಾನ ೧: ಪದ ಸಂಖ್ಯೆಗಳು + Padding + Dense

೧೨. ವಿಧಾನ ೨: One-hot (Multi-hot) ಎನ್‌ಕೋಡಿಂಗ್

೧೩. ವಿಧಾನ ೩: Word Embeddings

೧೪. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

೧. ದೊಡ್ಡ ಚಿತ್ರ: ಈ ಉಪನ್ಯಾಸದ ನಕ್ಷೆ

ಹಿಂದಿನ ಉಪನ್ಯಾಸಗಳಲ್ಲಿ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯುವುದು ಮತ್ತು ಅದರ ಕಾರ್ಯಕ್ಷಮತೆ ಅಳೆಯುವುದನ್ನು ನೋಡಿದೆವು. ಒಂದು ದೊಡ್ಡ ತೊಂದರೆ ಉಳಿದಿದೆ: **ಮಾದರಿ ತರಬೇತಿ ದತ್ತಾಂಶವನ್ನು ಉರು ಹೊಡೆಯುತ್ತದೆ (overfitting)**. ಈ ಉಪನ್ಯಾಸ ಅದನ್ನು ನಿಯಂತ್ರಿಸುವ ವಿಧಾನಗಳನ್ನು ಮತ್ತು ಪಠ್ಯ (text) ದತ್ತಾಂಶವನ್ನು ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಕೊಡುವ ವಿಧಾನಗಳನ್ನು ವಿವರಿಸುತ್ತದೆ.

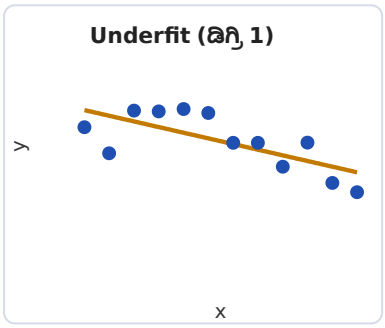
ಭಾಗ	ವಿಷಯ	ಒಂದು ಸಾಲಿನ ಸಾರ
ಭಾಗ ೧: ಮಾದರಿ ನಿಯಂತ್ರಣ	L1, L2, Elastic Net, Dropout, Batch Norm	ತೂಕಗಳು ಅತಿ ದೊಡ್ಡದಾಗದಂತೆ, ನ್ಯೂರಾನ್‌ಗಳು ಅತಿ ನಿಪುಣರಾಗದಂತೆ ತಡೆಯುವುದು
ಭಾಗ ೨: ಮೌಲ್ಯಮಾಪನ ತಂತ್ರಗಳು	Hold-out, K-Fold, Stratified, Shuffle, LOO, TimeSeries	ಮಾದರಿಯ ನಿಜವಾದ ಸಾಮರ್ಥ್ಯವನ್ನು ನಂಬಿಕೆಯಿಂದ ಅಳೆಯುವುದು
ಭಾಗ ೩: ಪಠ್ಯ ಎನ್‌ಕೋಡಿಂಗ್	Tokenizer, ಸಂಖ್ಯಾ ಸೂಚಿ, One-hot, Embedding	ಪದಗಳನ್ನು ಸಂಖ್ಯೆಗಳಾಗಿ ಬದಲಿಸುವ ಮೂರು ದಾರಿಗಳು (IMDB ವಿಮರ್ಶೆಗಳು)

೨. Regularisation — ಮಾದರಿಗೆ “ಲಗಾಮು”

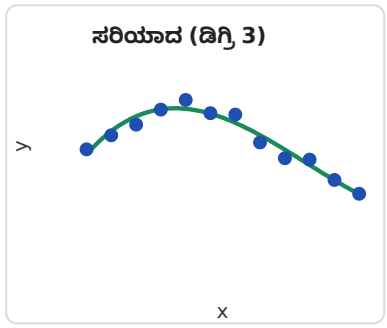
ಸವಾಲು	ಸಮಸ್ಯೆ	ಪರಿಹಾರ
ತರಬೇತಿ ದತ್ತಾಂಶದಿಂದ ಕಲಿಯುವುದು ಮತ್ತು ಹೊಸ ದತ್ತಾಂಶಕ್ಕೆ ಸಾಮಾನ್ಯೀಕರಿಸುವುದರ ನಡುವಿನ ಸಮತೋಲನ	**Overfitting:** ಮಾದರಿ ತರಬೇತಿ ದತ್ತಾಂಶವನ್ನು ಉರು ಹೊಡೆದು ಹೊಸ ಇನ್‌ಪುಟ್‌ನಲ್ಲಿ ವಿಫಲವಾಗುತ್ತದೆ	**Regularisation:** loss ಗೆ ಒಂದು **ದಂಡ (penalty)** ಸೇರಿಸಿ ದೊಡ್ಡ ತೂಕಗಳನ್ನು ನಿರುತ್ಸಾಹಗೊಳಿಸುವುದು; ಮಾದರಿ ಸರಳವಾಗುತ್ತದೆ

ಮಾದರಿ ಸಂಕೀರ್ಣತೆ ಮತ್ತು Overfitting

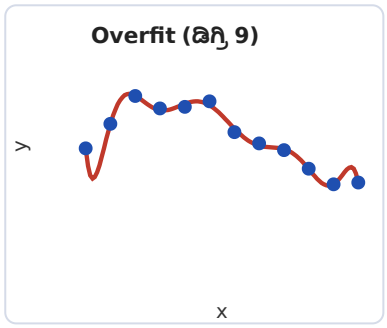
ಒಂದೇ ದತ್ತಾಂಶಕ್ಕೆ (12 ಬಿಂದುಗಳು, ಸ್ವಲ್ಪ noise ಸಹಿತ) ವಿವಿಧ ಸಂಕೀರ್ಣತೆಯ (polynomial degree) ವಕ್ರರೇಖೆ ಹೊಂದಿಸಿದಾಗ:



ತುಂಬ ಸರಳ — ಮಾದರಿಯನ್ನು ಹಿಡಿಯುವುದಿಲ್ಲ



ಮಧ್ಯಮ — ಒಟ್ಟಾರೆ ಪ್ರವೃತ್ತಿ ಹಿಡಿಯುತ್ತದೆ



ತುಂಬ ಸಂಕೀರ್ಣ — noise ಅನ್ನೂ ಹಿಂಬಾಲಿಸುತ್ತದೆ

ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಕೆಂಪು (ಡಿಗ್ರಿ 1) = underfitting, ನೀಲಿ (ಡಿಗ್ರಿ 2) = ಉತ್ತಮ fit, ಹಸಿರು (ಡಿಗ್ರಿ 300) = overfitting. ನಮ್ಮ ಚಿತ್ರಗಳಲ್ಲಿ ಡಿಗ್ರಿ 1, 3, 9 ಬಳಸಿದ್ದೇವೆ — ತತ್ವ ಒಂದೇ. ಹಸಿರು/ಕೆಂಪು ಬಿಂದುಗಳ ನಡುವೆ ವಕ್ರರೇಖೆ ಅಲೆದಾಡುವುದನ್ನು ನೋಡಿ.

> **ಹೋಲಿಕೆ: ವಿಮಾನದ ಲಗೇಜ್ ದಂಡ**
>
> ವಿಮಾನದಲ್ಲಿ ಹೆಚ್ಚು ಲಗೇಜ್‌ಗೆ ದಂಡ ಹಾಕುತ್ತಾರೆ. ಆಗ ಪ್ರಯಾಣಿಕ ಅನಗತ್ಯ ವಸ್ತುಗಳನ್ನು ಬಿಟ್ಟು ಅತ್ಯಗತ್ಯವಾದವನ್ನು ಮಾತ್ರ ಒಯ್ಯುತ್ತಾನೆ. Regularisation ಕೂಡ ಹಾಗೆ: ಮಾದರಿ ದೊಡ್ಡ ತೂಕಗಳನ್ನು ಬಳಸಿದರೆ “ದಂಡ” ಕಟ್ಟಬೇಕು. ಆದ್ದರಿಂದ ಅದು ಅತ್ಯಗತ್ಯ ಮಾದರಿಗಳನ್ನು ಮಾತ್ರ ಕಲಿಯುತ್ತದೆ, noise ಅನ್ನು ಉರು ಹೊಡೆಯುವುದಿಲ್ಲ.

ಮೂರು ಸಾಮಾನ್ಯ ವಿಧಾನಗಳು: Lasso (L1), Ridge/Tikhonov (L2), Elastic Net (L1 + L2).

೩. L2 (Ridge) Regularisation

L2 regularization (ಇದಕ್ಕೆ **Ridge**, **weight decay**, Tikhonov regularisation ಎಂದೂ ಹೆಸರು) loss ಗೆ **ತೂಕಗಳ ವರ್ಗಗಳ ಮೊತ್ತ**ಕ್ಕೆ ಅನುಪಾತದ ದಂಡ ಸೇರಿಸುತ್ತದೆ:

```
L_total = L_original + λ · Σ w_i²

L_total    = regularization ಸಹಿತ ಒಟ್ಟು loss
L_original = ಮೂಲ loss (ಉದಾ: MSE)
w_i        = ಮಾದರಿಯ ತೂಕಗಳು
λ (ಲ್ಯಾಂಬ್ಡಾ) = ದಂಡದ ಶಕ್ತಿ
```

- **λ ದೊಡ್ಡದು** → ದೊಡ್ಡ ತೂಕಗಳಿಗೆ ಹೆಚ್ಚು ದಂಡ → ಸರಳ ಮಾದರಿ (ಆದರೆ ತುಂಬ ದೊಡ್ಡದಾದರೆ underfitting).
- **λ ಚಿಕ್ಕದು** → ಹೆಚ್ಚು ಸ್ವಾತಂತ್ರ್ಯ (ಆದರೆ overfitting ಅಪಾಯ).
- Regularization **ತರಬೇತಿ ಸಮಯದಲ್ಲಿ ಮಾತ್ರ** ಕೆಲಸ ಮಾಡುತ್ತದೆ; ಪರೀಕ್ಷೆ/ಭವಿಷ್ಯವಾಣಿಯಲ್ಲಿ ಸ್ಥಿರ ತೂಕಗಳನ್ನು ಬಳಸುತ್ತೇವೆ.
- L2 ಸರಿಯಾಗಿ ಕೆಲಸ ಮಾಡಲು ತರಬೇತಿಗೆ ಮೊದಲು **features ಅನ್ನು normalise** ಮಾಡಿ (ಇಲ್ಲದಿದ್ದರೆ ದೊಡ್ಡ ಮಾಪಕದ feature ಗೆ ಅನ್ಯಾಯ).
- **λ ಆರಿಸುವ ವಿಧಾನ:** ಕೈಯಾರೆ (0.01, 0.1, 1), Grid search, ಅಥವಾ Cross-validation (ಉದಾ: scikit-learn ನ RidgeCV).

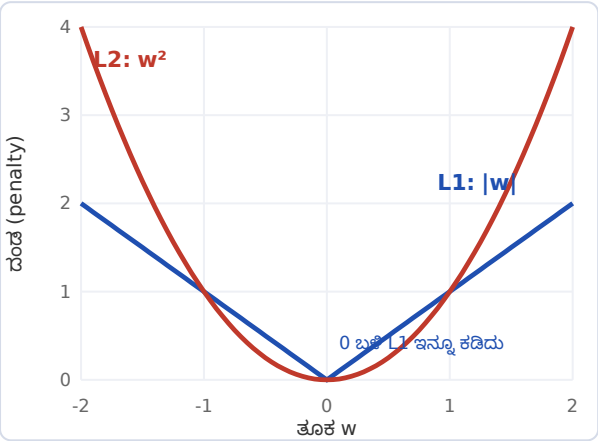
> **ಲೆಕ್ಕ: L2 ದಂಡ ಹೇಗೆ ಸೇರುತ್ತದೆ?**
>
> ಮಾದರಿಯ ತೂಕಗಳು w = [3, −2, 0.5]. ಮೂಲ loss = 0.40. λ = 0.1.
>
> ```
> Σ w² = 3² + (−2)² + 0.5² = 9 + 4 + 0.25 = 13.25
> ದಂಡ = λ × Σ w² = 0.1 × 13.25 = 1.325
> L_total = 0.40 + 1.325 = 1.725
> ```
>
> ದೊಡ್ಡ ತೂಕ 3 ಮಾತ್ರ (9) ಬಹುಪಾಲು ದಂಡ ಕೊಟ್ಟಿತು — ವರ್ಗ ಮಾಡುವುದರಿಂದ ದೊಡ್ಡ ತೂಕಗಳಿಗೆ ದಂಡ ಅತಿ ಹೆಚ್ಚು. ಈಗ ತೂಕ ಬದಲಾವಣೆಯನ್ನು ನೋಡೋಣ: ತೂಕ w = 3, ಡೇಟಾದಿಂದ ಬಂದ ಗ್ರೇಡಿಯಂಟ್ g = 0.5, ಕಲಿಕಾ ದರ η = 0.1.
>
> ```
> Regularization ಇಲ್ಲದೆ: w = 3 − 0.1 × 0.5            = 2.95
> L2 ಸಹಿತ     : w = 3 − 0.1 × (0.5 + 2×0.1×3) = 3 − 0.1 × 1.1 = 2.89
> ```
>
> L2 ಯ ಗ್ರೇಡಿಯಂಟ್ 2λw ಆಗಿರುವುದರಿಂದ ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ ತೂಕ ತನ್ನ ಮೌಲ್ಯದ ಅನುಪಾತದಲ್ಲಿ ಸ್ವಲ್ಪ ಕುಗ್ಗುತ್ತದೆ — ಇದೇ ಕಾರಣಕ್ಕೆ **weight decay** (ತೂಕ ಕ್ಷಯ) ಎನ್ನುತ್ತಾರೆ.

೪. L1 (Lasso) Regularisation ಮತ್ತು L1 vs L2 ಹೋಲಿಕೆ

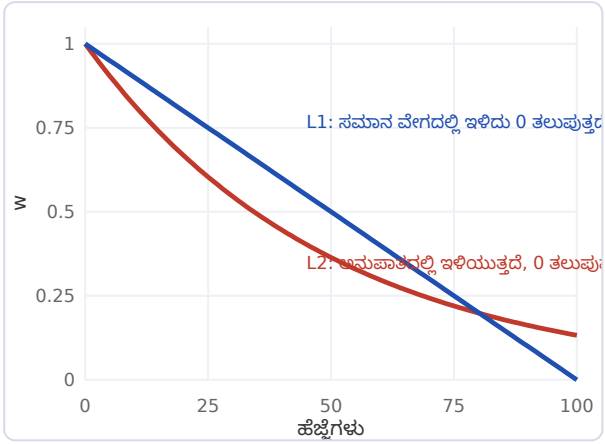
Lasso (Least Absolute Shrinkage and Selection Operator) ತೂಕಗಳ **ಸಂಪೂರ್ಣ ಮೌಲ್ಯಗಳ ಮೊತ್ತ**ಕ್ಕೆ ದಂಡ ಹಾಕುತ್ತದೆ. ಇದು ಕೆಲವು ತೂಕಗಳನ್ನು **ನಿಖರವಾಗಿ 0** ಮಾಡುತ್ತದೆ — ಮಾದರಿ "ವಿರಳ" (sparse) ಆಗುತ್ತದೆ ಮತ್ತು ಸ್ವಯಂಚಾಲಿತ **feature selection** ಆಗುತ್ತದೆ. ಬಹಳ features ಇದ್ದು ಕೆಲವು ಮಾತ್ರ ಉಪಯುಕ್ತವಾಗಿದ್ದಾಗ L1 ಒಳ್ಳೆಯದು.

```
L_total = L_original + λ · Σ |w_i|
```

- λ ದೊಡ್ಡದು → ಹೆಚ್ಚು ತೂಕಗಳು 0 (ಹೆಚ್ಚು sparsity) ಆದರೆ underfitting ಅಪಾಯ.
- ಸಾಮಾನ್ಯ ಮೌಲ್ಯಗಳು: 0.001 – 0.1 (ಕೈಯಾರೆ), ಅಥವಾ grid search / cross-validation.
- L1 ಗೂ ಮೊದಲು features ಅನ್ನು normalise ಮಾಡಿ.



ಚಿತ್ರ ೧: ದಂಡದ ಆಕಾರ — 0 ಹತ್ತಿರ L2 ಸಮತಟ್ಟು, L1 ಕಡಿದು



ಚಿತ್ರ ೨: ಕೇವಲ ದಂಡದ ಪ್ರಭಾವದಲ್ಲಿ w = 1 ಹೇಗೆ ಕುಗ್ಗುತ್ತದೆ

ಲೆಕ್ಕ: L1 ಏಕೆ ತೂಕವನ್ನು 0 ಮಾಡುತ್ತದೆ, L2 ಏಕೆ ಮಾಡುವುದಿಲ್ಲ?

ಡೇಟಾ ಗ್ರೇಡಿಯಂಟ್ ಇಲ್ಲವೆಂದು ಭಾವಿಸೋಣ (ಕೇವಲ ದಂಡ ಕೆಲಸ ಮಾಡುತ್ತಿದೆ), η = 0.1, λ = 0.1.

```
L2: ಪ್ರತಿ ಹೆಜ್ಜೆ  w ← w × (1 − 2ηλ) = w × 0.98        (ಅನುಪಾತದಲ್ಲಿ ಕುಗ್ಗುತ್ತದೆ)
L1: ಪ್ರತಿ ಹೆಜ್ಜೆ  w ← w − ηλ·sign(w) = w − 0.01      (ಸ್ಥಿರ ಪ್ರಮಾಣದಲ್ಲಿ ಕಡಿಮೆ)
```

ಹೆಜ್ಜೆಗಳು	ದೊಡ್ಡ ತೂಕ (w = 1): L2	L1	ಚಿಕ್ಕ ತೂಕ (w = 0.05): L2	L1
5	0.904	0.95	0.0452	**0** ✓
50	0.364	0.50	0.0182	**0**
100	0.133	**0** ✓	0.0066	**0**

L2 ನಲ್ಲಿ ತೂಕ ಚಿಕ್ಕದಾದಂತೆ ಕುಗ್ಗುವಿಕೆಯೂ ಚಿಕ್ಕದಾಗುತ್ತದೆ — 0 ತಲುಪುವುದೇ ಇಲ್ಲ. L1 ನಲ್ಲಿ ತೂಕ ಎಷ್ಟೇ ಚಿಕ್ಕದಿದ್ದರೂ ಅದೇ ಸ್ಥಿರ ಹೆಜ್ಜೆಯಲ್ಲಿ ಇಳಿದು ನಿಖರವಾಗಿ 0 ತಲುಪುತ್ತದೆ. ಹಾಗಾಗಿ ಅಪ್ರಸ್ತುತ features ನ ತೂಕ ಸೊನ್ನೆಯಾಗುತ್ತದೆ.

ಉದಾಹರಣೆ: ಮನೆ ಬೆಲೆ ಊಹೆ

ನಿಮ್ಮ ಬಳಿ 50 features ಇವೆ: ವಿಸ್ತೀರ್ಣ, ಕೊಠಡಿಗಳು, ಸ್ಥಳ… ಜೊತೆಗೆ ಬೀದಿಯ ಹೆಸರಿನ ಅಕ್ಷರ ಸಂಖ್ಯೆ, ಗೇಟ್‌ನ ಬಣ್ಣ ಮುಂತಾದ ಅಪ್ರಸ್ತುತ ಅಂಶಗಳು. L1 (Lasso) ಅಪ್ರಸ್ತುತ features ನ ತೂಕವನ್ನು 0 ಮಾಡಿ “ಇವು ಬೇಡ” ಎಂದು ಹೇಳುತ್ತದೆ. L2 (Ridge) ಯಾವುದನ್ನೂ ತೆಗೆಯದೆ ಎಲ್ಲ ತೂಕಗಳನ್ನು ಚಿಕ್ಕದಾಗಿ ಹಂಚುತ್ತದೆ.

L1 ಮತ್ತು L2 ಹೋಲಿಕೆ (ಸ್ಲೈಡ್‌ನ ಕೋಷ್ಟಕ)

	L1 (Lasso)	L2 (Ridge)
ಅನುಕೂಲಗಳು	Feature selection: ಅಪ್ರಸ್ತುತ features ನ ತೂಕ 0	ತೂಕಗಳನ್ನು ಸಮತೋಲಿತವಾಗಿ ಹಂಚುತ್ತದೆ; ಒಂದೇ feature ಮೇಲೆ ಅತಿಯಾದ ಅವಲಂಬನೆ ಕಡಿಮೆ
	Outliers ಗೆ ಕಡಿಮೆ ಸೂಕ್ಷ್ಮ (ಸಂಪೂರ್ಣ ಮೌಲ್ಯದ ದಂಡ)	ಪರಸ್ಪರ ಸಂಬಂಧಿತ (multicollinear) features ಗೆ ಒಳ್ಳೆಯದು — ತೂಕ ಸಮನಾಗಿ ಹಂಚುತ್ತದೆ
	ಸರಳ, ಅರ್ಥಮಾಡಿಕೊಳ್ಳಲು ಸುಲಭ ಮಾದರಿ	ಹೆಚ್ಚು ಸ್ಥಿರ (correlated ಇದ್ದರೂ)
ಅನಾನುಕೂಲಗಳು	0 ಯಲ್ಲಿ differentiable ಅಲ್ಲ — ವಿಶೇಷ optimization ವಿಧಾನ ಬೇಕು	ಯಾವ ತೂಕವನ್ನೂ ನಿಖರ 0 ಮಾಡುವುದಿಲ್ಲ → feature selection ಇಲ್ಲ
	ಗುಣಾಂಕಗಳನ್ನು ಅತಿಯಾಗಿ ಕುಗ್ಗಿಸಿ underfitting ಉಂಟುಮಾಡಬಹುದು	Outliers ಗೆ ಸೂಕ್ಷ್ಮ (ವರ್ಗದ ದಂಡ)
	ಹೆಚ್ಚು correlated features ಇದ್ದರೆ ಒಂದನ್ನು ಯಾದೃಚ್ಛಿಕವಾಗಿ ಆರಿಸಬಹುದು	ಎಲ್ಲ features ಉಳಿಯುವುದರಿಂದ ವ್ಯಾಖ್ಯಾನಿಸಲು ಕಷ್ಟ

೫. Elastic Net ಮತ್ತು Keras ನಲ್ಲಿ regularizers

Elastic Net = L1 ಮತ್ತು L2 ಎರಡರ ಮಿಶ್ರಣ. L1 ನ ಸರಳತೆ (sparsity) ಮತ್ತು L2 ನ ಸ್ಥಿರತೆ ಎರಡೂ ಸಿಗುತ್ತವೆ. ಮಿಶ್ರಣ ಪ್ರಮಾಣ α (0 ≤ α ≤ 1) ನಿಯಂತ್ರಿಸುತ್ತದೆ: **α = 1 → ಶುದ್ಧ Lasso (L1)**, **α = 0 → ಶುದ್ಧ Ridge (L2)**, ನಡುವೆ → ಎರಡರ ಮಿಶ್ರಣ.

```
L_total = L_original + λ · [ α · Σ|w_i| + (1 − α) · Σ w_i² ]
```

> **ಸ್ಲೈಡ್‌ನ ಸೂತ್ರದ ಬಗ್ಗೆ ಒಂದು ಎಚ್ಚರಿಕೆ**
> ಸ್ಲೈಡ್‌ನ ಮುದ್ರಿತ ಸೂತ್ರದಲ್ಲಿ (1 − α) L1 ಭಾಗಕ್ಕೂ ಮತ್ತು α L2 ಭಾಗಕ್ಕೂ ಜೋಡಣೆಯಾಗಿದೆ. ಆದರೆ ಅದರ ಕೆಳಗಿನ ವಿವರಣೆ "α = 1 → Lasso" ಎನ್ನುತ್ತದೆ — ಇವೆರಡು ಹೊಂದಿಕೆಯಾಗುವುದಿಲ್ಲ. ವಿವರಣೆಗೆ ಹೊಂದುವಂತೆ ಮೇಲೆ ಸರಿಪಡಿಸಿದ ಸೂತ್ರ ನೀಡಿದ್ದೇವೆ (ಇದು ಸಾಮಾನ್ಯವಾಗಿ ಬಳಸುವ ಸಂಪ್ರದಾಯ).

ಲೆಕ್ಕ: ಅದೇ ತೂಕಗಳು w = [3, −2, 0.5], λ = 0.1

```
L1 ಮಾತ್ರ (α = 1):  0.1 × (3 + 2 + 0.5)                    = 0.1 × 5.5
= 0.550
L2 ಮಾತ್ರ (α = 0):  0.1 × (9 + 4 + 0.25)                     = 0.1 ×
13.25 = 1.325
ಮಿಶ್ರ (α = 0.5):   0.1 × (0.5 × 5.5 + 0.5 × 13.25)           = 0.1 × 9.375
= 0.9375
```

Keras ನಲ್ಲಿ ಬಳಕೆ (ನನ್ನ ಸೇರ್ಪಡೆ — ಸ್ಲೈಡ್ ಕೇವಲ keras.io/api/layers/regularizers ಲಿಂಕ್ ನೀಡುತ್ತದೆ)

```
from keras import regularizers
from keras.layers import Dense

Dense(100, activation='relu',
      kernel_regularizer=regularizers.l2(0.01))        # L2: λ = 0.01
Dense(100, activation='relu',
      kernel_regularizer=regularizers.l1(0.001))       # L1: λ = 0.001
Dense(100, activation='relu',
      kernel_regularizer=regularizers.l1_l2(l1=0.001, l2=0.01))   # Elastic Net
```

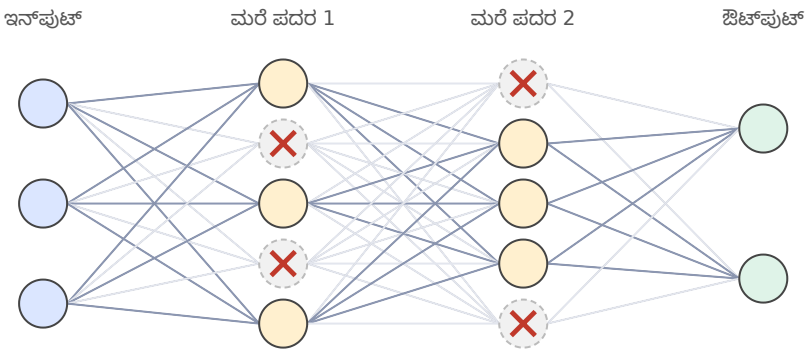
೬. Early Stopping (ಪುನರಾವಲೋಕನ) ಮತ್ತು Dropout

Early Stopping

ಓವರ್‌ಫಿಟ್ಟಿಂಗ್ ತಡೆಯಲು ಒಂದು ಸರಳ ವಿಧಾನ: validation error ಏರಲು ಆರಂಭಿಸಿದಾಗ ತರಬೇತಿ ನಿಲ್ಲಿಸಿ ಅತ್ಯುತ್ತಮ ಮಾದರಿಯನ್ನು ಉಳಿಸುವುದು (ಸ್ಲೈಡ್‌ನ ಚಿತ್ರದಲ್ಲಿ “Best model” ಬಿಂದು). ಹಿಂದಿನ ಉಪನ್ಯಾಸದಲ್ಲಿ ವಿವರವಾಗಿ ನೋಡಿದ್ದೇವೆ.

Dropout — ನ್ಯೂರಾನ್‌ಗಳು ಅತಿ ನಿಪುಣರಾಗದಂತೆ ತಡೆಯುವುದು

ಗುರಿ: **ಯಾವುದೇ ನ್ಯೂರಾನ್ ಅತಿಯಾಗಿ ಪರಿಣತ (specialised) ಆಗದಂತೆ ತಡೆಯುವುದು.** ತರಬೇತಿಯ ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ ಯಾದೃಚ್ಛಿಕವಾಗಿ ಕೆಲವು ನ್ಯೂರಾನ್‌ಗಳನ್ನು **ತಾತ್ಕಾಲಿಕವಾಗಿ ಆಫ್** ಮಾಡುತ್ತೇವೆ.



ಕೆಂಪು × = ಈ ಹೆಜ್ಜೆಯಲ್ಲಿ ಯಾದೃಚ್ಛಿಕವಾಗಿ ಆಫ್ ಮಾಡಿದ ನ್ಯೂರಾನ್‌ಗಳು (dropout)

ಚಿತ್ರ ೩: ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ ಬೇರೆ ಬೇರೆ ನ್ಯೂರಾನ್‌ಗಳು ಆಫ್ ಆಗುತ್ತವೆ

ಕಥೆ: ಆಸ್ಪತ್ರೆಯ ವೈದ್ಯರ ತಂಡ ಮತ್ತು ಕ್ರಿಕೆಟ್ ತಂಡ

ಒಂದು ಆಸ್ಪತ್ರೆಯಲ್ಲಿ ಹೃದ್ರೋಗ, ಮನೋರೋಗ, ಮಧುಮೇಹ, ಕ್ಯಾನ್ಸರ್ ಮುಂತಾದ ಪರಿಣತ ವೈದ್ಯರಿದ್ದಾರೆ (ಸ್ಲೈಡ್‌ನ ಚಿತ್ರ). ಪ್ರತಿ ಬಾರಿ ಒಬ್ಬನೇ ಪ್ರಸಿದ್ಧ ವೈದ್ಯನ ಮೇಲೆ ಎಲ್ಲರೂ ಅವಲಂಬಿತರಾದರೆ, ಆತ ರಜೆ ಹಾಕಿದಾಗ ಇಡೀ ತಂಡ ಕುಸಿಯುತ್ತದೆ. ಹಾಗೆಯೇ ಕ್ರಿಕೆಟ್ ತಂಡ ಒಬ್ಬ ಸ್ಟಾರ್ ಬ್ಯಾಟ್ಸ್‌ಮನ್ ಮೇಲೆ ಮಾತ್ರ ನಿಂತರೆ ಅವನು ಔಟ್ ಆದ ತಕ್ಷಣ ಸೋಲುತ್ತದೆ. ಪರಿಹಾರ: ಅಭ್ಯಾಸದಲ್ಲಿ ಯಾದೃಚ್ಛಿಕವಾಗಿ ಕೆಲವರಿಗೆ ವಿಶ್ರಾಂತಿ ಕೊಡಿ — ಉಳಿದವರು ಜವಾಬ್ದಾರಿ ಹಂಚಿಕೊಂಡು ಎಲ್ಲರೂ ಸಮರ್ಥರಾಗುತ್ತಾರೆ. Dropout ನಲ್ಲಿ ನ್ಯೂರಾನ್‌ಗಳೂ ಹೀಗೆಯೇ ಒಬ್ಬರನ್ನೊಬ್ಬರು ಅವಲಂಬಿಸದೆ ಸ್ವತಂತ್ರವಾಗಿ ಉಪಯುಕ್ತ ಲಕ್ಷಣ ಕಲಿಯುತ್ತವೆ.

ಸಂಕೇತ	ಅರ್ಥ
rate = p	ನ್ಯೂರಾನ್ ಆಫ್ ಆಗುವ ಸಂಭವನೀಯತೆ (ಉದಾ: 0.4)
q = 1 − p	ನ್ಯೂರಾನ್ ಸಕ್ರಿಯವಾಗಿ ಉಳಿಯುವ ಸಂಭವನೀಯತೆ
S	Dropout ಇಲ್ಲದ ಒಟ್ಟು ಇನ್‌ಪುಟ್ ಮೊತ್ತ: $S = x_1w_1 + x_2w_2 + \ldots + x_7w_7$
S_q	Dropout ನಂತರ ಉಳಿದ (ಸಕ್ರಿಯ) ನ್ಯೂರಾನ್‌ಗಳ ಮೊತ್ತ

ಸ್ಕೇಲಿಂಗ್ — ಸಂಕೇತ ಸಮತೋಲನದಲ್ಲಿರಲು

ಕೆಲವು ನ್ಯೂರಾನ್‌ಗಳನ್ನು ಆಫ್ ಮಾಡಿದರೆ ಪದರದ ಔಟ್‌ಪುಟ್ ಸಹಜವಾಗಿ ಚಿಕ್ಕದಾಗುತ್ತದೆ. ಅದನ್ನು ಸರಿದೂಗಿಸಲು ಸಕ್ರಿಯ ನ್ಯೂರಾನ್‌ಗಳ ಮೌಲ್ಯವನ್ನು **1/q = 1/(1 − p) = 1/(1 − rate)** ರಿಂದ ಗುಣಿಸುತ್ತೇವೆ, ಆಗ ಒಟ್ಟು ಮೊತ್ತ ಸರಾಸರಿಯಲ್ಲಿ ಬದಲಾಗುವುದಿಲ್ಲ: $\hat{S}_q = S_q / q \approx S$.

ಲೆಕ್ಕ: 7 ಇನ್‌ಪುಟ್‌ಗಳು, rate = 0.4

x = [1, 2, 1, 3, 2, 1, 2], w = [0.5, 0.2, 0.4, 0.1, 0.3, 0.6, 0.1]

```
ಪೂರ್ಣ ಮೊತ್ತ  S = 0.5 + 0.4 + 0.4 + 0.3 + 0.6 + 0.6 + 0.2 = 3.0

ನ್ಯೂರಾನ್ 3, 6, 7 ಆಫ್ ಆದವು (ಸ್ಲೈಡ್‌ನ ಸೂತ್ರ (2) ನಂತೆ x₁,x₂,x₄,x₅ ಮಾತ್ರ ಉಳಿದವು):
S_q = x₁w₁ + x₂w₂ + x₄w₄ + x₅w₅ = 0.5 + 0.4 + 0.3 + 0.6 = 1.8   (ಚಿಕ್ಕದಾಯಿತು!)

ಸ್ಕೇಲಿಂಗ್: 1/(1 − 0.4) = 1/0.6 = 1.667
Ŝ_q = 1.8 × 1.667 = 3.0   ✓  (ಮೂಲ S ಗೆ ಸಮ)
```

ಮುಖ್ಯ ನಿಯಮಗಳು

• Dropout **ತರಬೇತಿಯಲ್ಲಿ ಮಾತ್ರ** ಸಕ್ರಿಯ; ಭವಿಷ್ಯವಾಣಿ (predict/evaluate) ಸಮಯದಲ್ಲಿ ಎಲ್ಲ ನ್ಯೂರಾನ್‌ಗಳೂ ಇರುತ್ತವೆ (Keras ತಾನಾಗಿ ನೋಡಿಕೊಳ್ಳುತ್ತದೆ).
• ಸ್ಕೇಲಿಂಗ್ 1/(1 − rate) ಅನ್ನು ತರಬೇತಿ ಸಮಯದಲ್ಲೇ Keras ಅನ್ವಯಿಸುತ್ತದೆ.
• ಸಾಮಾನ್ಯ rate ಮೌಲ್ಯಗಳು 0.2 – 0.5 (ನನ್ನ ಟಿಪ್ಪಣಿ).

Keras ನಲ್ಲಿ (ಸ್ಲೈಡ್ 10 ರ ಕೋಡ್)

```
from keras.layers import Dense, Flatten, Dropout, Input

model = keras.Sequential([
    Input(shape=(28, 28, 1)),
    Flatten(),
    Dense(100, activation='sigmoid'),   # ಮರೆ ಪದರ (ಸಕ್ರಿಯಕರಣ: sigmoid, tanh, elu, selu,
relu)
    Dropout(0.4),                       # ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ 40% ನ್ಯೂರಾನ್‌ಗಳನ್ನು ಯಾದೃಚ್ಛಿಕವಾಗಿ ಆಫ್
    Dense(10, activation='softmax')     # ಔಟ್‌ಪುಟ್ ಪದರ
])
print(model.summary())
```

೭. Batch Normalization

ಕಲ್ಪನೆ: ಪ್ರತಿ mini-batch ನಲ್ಲಿ ಪದರದ ಆಕ್ಟಿವೇಶನ್‌ಗಳನ್ನು ಸರಾಸರಿ 0, ವ್ಯತ್ಯಾಸ 1 ಇರುವಂತೆ ಮರುಮಾಪನ (normalise) ಮಾಡುವುದು. ಇದು ತರಬೇತಿಯನ್ನು ಸ್ಥಿರ ಮತ್ತು ವೇಗ ಮಾಡುತ್ತದೆ ಹಾಗೂ ಸ್ವಲ್ಪ regularization ಪರಿಣಾಮವನ್ನೂ ಕೊಡುತ್ತದೆ.

ಹಂತ	ಸೂತ್ರ
೧. mini-batch ನ ಸರಾಸರಿ ಮತ್ತು ವ್ಯತ್ಯಾಸ	μ_B, σ^2_B
೨. normalise (ε = ಸೊನ್ನೆಯಿಂದ ಭಾಗಿಸದಂತೆ ಚಿಕ್ಕ ಸಂಖ್ಯೆ)	$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$
೩. ಕಲಿಯಬಲ್ಲ ಮಾಪನ γ ಮತ್ತು ಸರಿಸುವಿಕೆ β	$y_i = \gamma \cdot \hat{x}_i + \beta$

ಲೆಕ್ಕ: ಒಂದು ನ್ಯೂರಾನ್‌ನ 4 ಮೌಲ್ಯಗಳ mini-batch: [2, 4, 6, 8]

$$\mu_B = (2 + 4 + 6 + 8) / 4 = 5$$
$$\sigma^2_B = ((-3)^2 + (-1)^2 + 1^2 + 3^2) / 4 = (9 + 1 + 1 + 9) / 4 = 5$$
$$\hat{x} = (x - 5) / \sqrt{5} \rightarrow [-1.342, -0.447, +0.447, +1.342] \quad (\varepsilon \approx 0)$$

$\gamma = 2$, $\beta = 1$ ಆದರೆ: $y = 2 \cdot \hat{x} + 1 \rightarrow [-1.683, 0.106, 1.894, 3.683]$

ಮೊದಲು ಮೌಲ್ಯಗಳು 2 ರಿಂದ 8 ರ ವ್ಯಾಪ್ತಿಯಲ್ಲಿ ಇದ್ದವು; normalise ನಂತರ -1.3 ರಿಂದ $+1.3$. γ, β ತರಬೇತಿಯಲ್ಲಿ ಕಲಿಯಲ್ಪಡುತ್ತವೆ — ಅಂದರೆ ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಅಗತ್ಯವಿದ್ದರೆ ಮೂಲ ಮಾಪನಕ್ಕೆ ಹಿಂತಿರುಗುವ ಸ್ವಾತಂತ್ರ್ಯವೂ ಇದೆ.

ಹೋಲಿಕೆ: ವಿವಿಧ ಶಾಲೆಗಳ ಅಂಕಗಳನ್ನು ಒಂದೇ ಮಾಪಕಕ್ಕೆ ತರುವುದು

ಒಂದು ಶಾಲೆಯಲ್ಲಿ 100 ರಲ್ಲಿ, ಇನ್ನೊಂದರಲ್ಲಿ 20 ರಲ್ಲಿ ಅಂಕ ಕೊಡುತ್ತಾರೆ. ಎರಡನ್ನು ಹೋಲಿಸಲು ಎರಡನ್ನೂ ಒಂದೇ ಮಾಪಕಕ್ಕೆ ತರಬೇಕು. Batch Norm ಪ್ರತಿ ಪದರದ ಸಂಕೇತಗಳನ್ನು ಹೀಗೆ ಸಮಾನ ಮಾಪಕಕ್ಕೆ ತರುತ್ತದೆ; ಹಿಂದಿನ ಪದರದ ಮೌಲ್ಯ ಬದಲಾದರೂ ಮುಂದಿನ ಪದರಕ್ಕೆ ಅದೇ ವ್ಯಾಪ್ತಿಯ ಸಂಖ್ಯೆಗಳು ಬರುತ್ತವೆ.

```
from keras.layers import Dense, BatchNormalization, Activation

model = keras.Sequential([
    Input(shape=(784,)),
    Dense(64),                  # ಇನ್ನೂ activation ಇಲ್ಲ
    BatchNormalization(),       # mini-batch normalise (γ, β ಕಲಿಯಲ್ಪಡುತ್ತವೆ)
    Activation('relu'),         # ನಂತರ activation
    Dense(10, activation='softmax')
])
# γ, β ಬೇಡವಾದರೆ: BatchNormalization(center=False, scale=False)
```

(ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಇದೇ ಬಗೆಯ ಕೋಡ್ ತೋರಿಸಲಾಗಿದೆ — Dense, ನಂತರ BatchNormalization, ನಂತರ Activation. ಮೇಲಿನ ಕೋಡ್ ತತ್ವ ವಿವರಿಸಲು ನಮ್ಮದೇ ಸರಳ ರೂಪ.)

೮. Validation ತಂತ್ರಗಳು: Hold-out ಮತ್ತು K-Fold

ಮಾದರಿಯ ನಿಜವಾದ ಸಾಮರ್ಥ್ಯ ಅಳೆಯಲು ದತ್ತಾಂಶ ವಿಭಜಿಸುವ ವಿಧಾನಗಳು ಹಲವು. ಸ್ಲೈಡ್‌ನ ಸಾರಾಂಶ ಕೋಷ್ಟಕ:

ವಿಧಾನ	ಸಂಕ್ಷಿಪ್ತ ವಿವರ	ಯಾವ ಮಾದರಿಗಳಲ್ಲಿ	ದತ್ತಾಂಶ ಗಾತ್ರ
Hold-out	ಒಮ್ಮೆ train/validation/test ವಿಭಜನೆ	ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್, boosting, SVM, trees	ಯಾವುದೇ, ವಿಶೇಷವಾಗಿ ದೊಡ್ಡದು
K-fold	K ಭಾಗ; ಪ್ರತಿ ಭಾಗ ಒಮ್ಮೆ validation	SVM, trees, logistic regression	ಸಣ್ಣ–ಮಧ್ಯಮ
Stratified K-fold	ಪ್ರತಿ fold ನಲ್ಲಿ ವರ್ಗ ಅನುಪಾತ ಉಳಿಯುತ್ತದೆ	ಅಸಮತೋಲನ ವರ್ಗೀಕರಣ	ಸಣ್ಣ–ಮಧ್ಯಮ
Leave-One-Out	ಒಂದು ಮಾದರಿ validation, ಉಳಿದವು train	ಸರಳ ಮಾದರಿ, ಅತಿ ಸಣ್ಣ ದತ್ತಾಂಶ	ಅತಿ ಸಣ್ಣ
Time Series Split	ಕಾಲಕ್ರಮ ಗೌರವಿಸಿ: ಹಿಂದಿನದು train, ಮುಂದಿನದು validation	RNN, LSTM, ಕಾಲ-ಸರಣಿ	ಯಾವುದೇ (ಕಾಲಕ್ರಮ ಇರಬೇಕು)
Shuffle Split	ಹಲವು ಯಾದೃಚ್ಛಿಕ ವಿಭಜನೆಗಳು	ಶಾಲೋ ಮಾದರಿಗಳು	ಸಣ್ಣ–ಮಧ್ಯಮ
Leave-P-Out	ಪ್ರತಿ ಬಾರಿ P ಮಾದರಿಗಳು test	ಸರಳ ಮಾದರಿ, ಅತಿ ಸಣ್ಣ ದತ್ತಾಂಶ	ಅತಿ ಸಣ್ಣ

Hold-out (ಸರಳ ವಿಭಜನೆ) ಮತ್ತು validation_split

- **Training set** — ತರಬೇತಿಗೆ; **Validation set** — hyperparameters ಟ್ಯೂನ್ ಮಾಡಲು ಮತ್ತು overfitting ಗಮನಿಸಲು; **Test set** — ಅಂತಿಮ ಮೌಲ್ಯಮಾಪನಕ್ಕೆ ಮಾತ್ರ (ಪ್ರತ್ಯೇಕವಾಗಿ ಇರಿಸಿ!).
- Keras ನಲ್ಲಿ `validation_split=0.2` ಕೊಟ್ಟರೆ 20% ದತ್ತಾಂಶ ತಾನಾಗಿ validation ಗೆ ಮೀಸಲಾಗುತ್ತದೆ.
- ಡೀಫಾಲ್ಟ್ ಆಗಿ ದತ್ತಾಂಶ ಕಲಸಲ್ಪಡುತ್ತದೆ (`shuffle=True`). ಕಲಸಬಾರದಾದರೆ `shuffle=False` .

ಯಾವಾಗ shuffling ನಿಲ್ಲಿಸಬೇಕು?

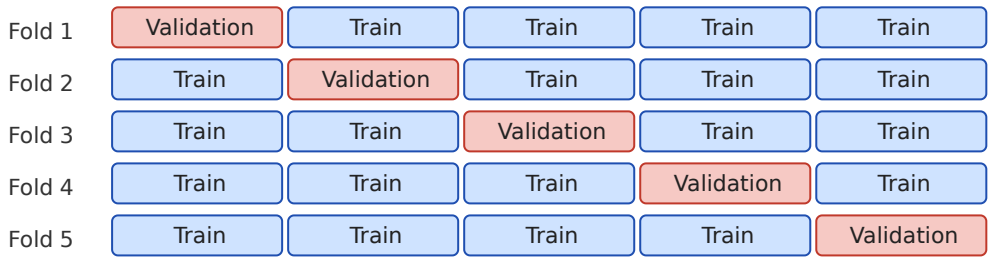
ದತ್ತಾಂಶದ ಕ್ರಮಕ್ಕೆ ಅರ್ಥವಿದ್ದರೆ (ಕಾಲ-ಸರಣಿ ಮುನ್ಸೂಚನೆ, ಸೆನ್ಸಾರ್ ಓದುಗಳು, ಈವೆಂಟ್ ಲಾಗ್‌ಗಳು). ಕಲಸಿದರೆ **ಭವಿಷ್ಯದ ಮಾಹಿತಿ ತರಬೇತಿಗೆ ಸೋರಿ ಹೋಗುತ್ತದೆ (data leakage)** ಮತ್ತು ಫಲಿತಾಂಶ ಸುಳ್ಳು ಒಳ್ಳೆಯದಾಗಿ ಕಾಣುತ್ತದೆ. *ಉದಾಹರಣೆ:* ನಾಳೆಯ ಷೇರು ಬೆಲೆ ಊಹಿಸುವ ಮಾದರಿಗೆ ಮುಂದಿನ ವಾರದ ಬೆಲೆಯನ್ನು ತರಬೇತಿಯಲ್ಲಿ ತೋರಿಸಿದರೆ ಅದು ಅದ್ಭುತವಾಗಿ ಕಾಣುತ್ತದೆ, ಆದರೆ ನಿಜ ಜೀವನದಲ್ಲಿ ವಿಫಲ.

ಹೋಲಿಕೆ: ಪ್ರಶ್ನೆಪತ್ರಿಕೆ ಸೋರಿಕೆ

ಪರೀಕ್ಷೆಯ ಪ್ರಶ್ನೆಪತ್ರಿಕೆ ಮುಂಚೆ ಸಿಕ್ಕ ವಿದ್ಯಾರ್ಥಿ ಪಡೆಯುವ ಅಂಕ ಅವನ ನಿಜವಾದ ಸಾಮರ್ಥ್ಯವನ್ನು ತೋರಿಸುವುದಿಲ್ಲ — data leakage ಅದೇ.

K-Fold Cross-Validation

ದತ್ತಾಂಶವನ್ನು K ಸಮಾನ ಭಾಗಗಳಾಗಿ (folds) ವಿಭಜಿಸಿ, ಮಾದರಿಯನ್ನು K ಬಾರಿ ತರಬೇತಿ ಮಾಡುತ್ತೇವೆ — ಪ್ರತಿ ಬಾರಿ ಬೇರೆ ಒಂದು fold validation ಗೆ, ಉಳಿದವು ತರಬೇತಿಗೆ. ನಂತರ K ಫಲಿತಾಂಶಗಳ ಸರಾಸರಿ.



ಪ್ರತಿ ಸುತ್ತಿನಲ್ಲಿ ಬೇರೆ ಭಾಗವೇ validation

ಚಿತ್ರ ೪: 5-fold: ಪ್ರತಿ ಭಾಗವೂ ಒಮ್ಮೆ validation ಆಗುತ್ತದೆ

ಲೆಕ್ಕ: 5-fold ಫಲಿತಾಂಶ

ಐದು ಸುತ್ತುಗಳ validation accuracy: 0.90, 0.88, 0.92, 0.86, 0.94

```
ಸರಾಸರಿ = (0.90 + 0.88 + 0.92 + 0.86 + 0.94) / 5 = 4.50 / 5 = 0.90  → 90%
ಪ್ರಮಾಣಿತ ವಿಚಲನೆ ≈ 0.032  (ಅಂದಾಜು ಎಷ್ಟು ಸ್ಥಿರ ಎಂದು ತಿಳಿಸುತ್ತದೆ)
```

ಒಂದೇ ವಿಭಜನೆಯಲ್ಲಿ 0.94 (ಅದೃಷ್ಟ) ಅಥವಾ 0.86 (ದುರದೃಷ್ಟ) ಬರಬಹುದು; ಸರಾಸರಿ 0.90 ಹೆಚ್ಚು ನಂಬಲರ್ಹ.

ಯಾವಾಗ ಸೂಕ್ತ ✓	ಯಾವಾಗ ಸೂಕ್ತವಲ್ಲ ✗
ದತ್ತಾಂಶ ಸಣ್ಣ/ಮಧ್ಯಮ; ಒಂದು ವಿಭಜನೆಗೆ ದತ್ತಾಂಶ ಕಳೆದುಕೊಳ್ಳಲು ಸಾಧ್ಯವಿಲ್ಲ ಹಲವು ವಿಭಜನೆಗಳ ಮೇಲೆ ನ್ಯಾಯಯುತ ಮೌಲ್ಯಮಾಪನ ಬೇಕು ಸ್ಥಿರ, ಸರಾಸರಿ ಫಲಿತಾಂಶ ಬೇಕಾದಾಗ ಉತ್ತಮ ಮಾದರಿ/ಸೆಟ್ಟಿಂಗ್ ಆರಿಸಲು ವೇಗವಾಗಿ ತರಬೇತಿಯಾಗುವ classical ML (SVM, Random Forest, XGBoost)	**ಡೀಪ್ ಲರ್ನಿಂಗ್:** K ಬಾರಿ ತರಬೇತಿ ಬಹಳ ನಿಧಾನ, ಸಂಪನ್ಮೂಲ ದುಬಾರಿ **ಕಾಲ-ಸರಣಿ ದತ್ತಾಂಶ:** TimeSeriesSplit ಬಳಸಿ **ಅಸಮತೋಲನ ವರ್ಗಗಳು:** Stratified K-Fold ಬಳಸಿ ರೋಗಿ/ಬಳಕೆದಾರ ಮಟ್ಟದಲ್ಲಿ ಕಟ್ಟುನಿಟ್ಟಾದ ಬೇರ್ಪಡಿಕೆ ಬೇಕಾದಾಗ (leakage ತಪ್ಪಿಸಲು)

೯. Stratified, ShuffleSplit, Leave-P-Out, TimeSeriesSplit

Stratified K-Fold

K-Fold ನ ಒಂದು ರೂಪ — **ಪ್ರತಿ fold ನಲ್ಲಿ ವರ್ಗಗಳ ಅನುಪಾತ ಮೂಲ ದತ್ತಾಂಶದಂತೆಯೇ** ಉಳಿಯುತ್ತದೆ. ಅಸಮತೋಲನ ದತ್ತಾಂಶದಲ್ಲಿ ಅತ್ಯಗತ್ಯ. ಲಾಭ: (೧) ಕೇವಲ ಬಹುಸಂಖ್ಯಾತ ವರ್ಗ ಇರುವ fold ತಡೆಯುತ್ತದೆ, (೨) ಪ್ರತಿ fold ನಲ್ಲಿ ಎಲ್ಲ ವರ್ಗಗಳೂ ಇರುತ್ತವೆ, (೩) ಹೆಚ್ಚು ಸ್ಥಿರ ಮತ್ತು ನ್ಯಾಯಯುತ ಫಲಿತಾಂಶ.

ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆ: 12 ಮಾದರಿಗಳು, 2 ವರ್ಗಗಳು, 3 folds

ಮಾದರಿಗಳು [0…11]; ವರ್ಗ 0: ಮಾದರಿ 0–3 (4 = 33%), ವರ್ಗ 1: ಮಾದರಿ 4–11 (8 = 67%).

Fold	Validation ಮಾದರಿಗಳು	ವರ್ಗ ಹಂಚಿಕೆ
1	[0, 4, 5, 6]	ವರ್ಗ 0: 1, ವರ್ಗ 1: 3
2	[1, 7, 8]	ವರ್ಗ 0: 1, ವರ್ಗ 1: 2
3	[2, 3, 9, 10, 11]	ವರ್ಗ 0: 2, ವರ್ಗ 1: 3

ಪ್ರತಿ fold ನಲ್ಲಿ ವರ್ಗ 0 ಮತ್ತು 1 ರ ಅನುಪಾತ ಸುಮಾರು 33 / 67 ಉಳಿದಿದೆ. ಸಾಮಾನ್ಯ K-Fold ನಲ್ಲಿ ಒಂದು fold ನಲ್ಲಿ ವರ್ಗ 0 ಒಂದೂ ಇಲ್ಲದಿರಬಹುದು. ಗಮನಿಸಿ: **groups** ಎಂದರೆ ಒಂದೇ ವಸ್ತು/ಘಟನೆಗೆ ಸೇರಿದ ಮಾದರಿಗಳ ಲೇಬಲ್ (ಉದಾ: ಒಬ್ಬನೇ ರೋಗಿಯ ಹಲವು ಸ್ಕ್ಯಾನ್‌ಗಳು) — ಇವನ್ನು ಒಂದೇ ಬದಿಯಲ್ಲಿ ಇಡಲು ಬಳಸುತ್ತಾರೆ.

ShuffleSplit

ದತ್ತಾಂಶವನ್ನು ಯಾದೃಚ್ಛಿಕವಾಗಿ ಕಲಸಿ train/validation ಆಗಿ ವಿಭಜಿಸಿ, ಇದನ್ನು ಹಲವು ಬಾರಿ ಪುನರಾವರ್ತಿಸುತ್ತದೆ; ಸರಾಸರಿ ಫಲಿತಾಂಶ. ಸಣ್ಣ–ಮಧ್ಯಮ ದತ್ತಾಂಶಕ್ಕೆ, ವರ್ಗಗಳು ಸಮತೋಲನವಿದ್ದಾಗ ಉತ್ತಮ; train/test ಗಾತ್ರದ ಮೇಲೆ ನಿಖರ ನಿಯಂತ್ರಣ ಸಿಗುತ್ತದೆ.

ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆ: 10 ಮಾದರಿಗಳು, 3 ಯಾದೃಚ್ಛಿಕ ವಿಭಜನೆ, test_size = 0.3

ವಿಭಜನೆ	Training (7)	Validation (3)
1	[0, 1, 2, 4, 5, 7, 9]	[3, 6, 8]
2	[0, 2, 3, 6, 7, 8, 9]	[1, 4, 5]
3	[1, 2, 3, 5, 6, 8, 9]	[0, 4, 7]

ಮಿತಿಗಳು: ವರ್ಗ ಅನುಪಾತ ಉಳಿಯುವುದಿಲ್ಲ (ಅಪರೂಪದ ವರ್ಗ ಕೆಲವು ವಿಭಜನೆಗಳಲ್ಲಿ ಇಲ್ಲದಿರಬಹುದು); ಮಾದರಿ 4 ಎರಡು ಬಾರಿ validation ನಲ್ಲಿದೆ, ಮಾದರಿ 9 ಒಮ್ಮೆಯೂ ಇಲ್ಲ — ಕೆಲವು ಮಾದರಿಗಳು ಹೆಚ್ಚು ಬಾರಿ ಬರುತ್ತವೆ.

Leave-One-Out (LOOCV) ಮತ್ತು Leave-P-Out (LPOCV)

LOO: ಪ್ರತಿ ಬಾರಿ ಒಂದೇ ಮಾದರಿ test, ಉಳಿದ ಎಲ್ಲ train. **LPO:** ಪ್ರತಿ ಬಾರಿ P ಮಾದರಿಗಳು test, ಉಳಿದವು train — ಎಲ್ಲ ಸಂಯೋಜನೆಗಳಿಗೂ.

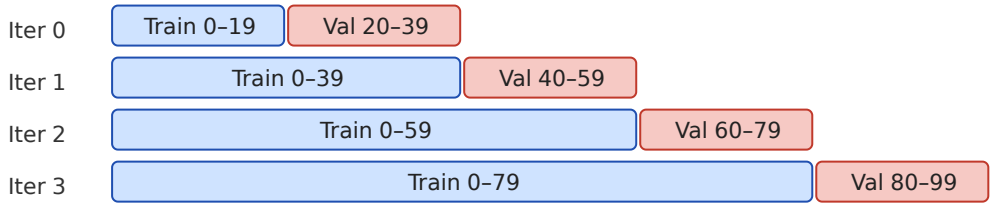
ಲೆಕ್ಕ: 100 ಮಾದರಿಗಳು (ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆ)

```
LOO           : 100 ಬಾರಿ ತರಬೇತಿ
LPO (p = 3)   : C(100, 3) = 100·99·98 / (3·2·1) = 161,700 ಬಾರಿ ತರಬೇತಿ !!
```

ಅತಿ ಸಣ್ಣ ದತ್ತಾಂಶದಲ್ಲಿ (ಎಲ್ಲ ದತ್ತಾಂಶ ಬಳಕೆ, ನಿಷ್ಪಕ್ಷಪಾತ ಅಂದಾಜು) ಒಳ್ಳೆಯದು; ಆದರೆ ದೊಡ್ಡ ದತ್ತಾಂಶ ಅಥವಾ ಸಂಕೀರ್ಣ ಮಾದರಿಗೆ ಅತಿ ನಿಧಾನ. ಡೀಪ್ ಲರ್ನಿಂಗ್‌ನಲ್ಲಿ ಬಳಸುವುದಿಲ್ಲ.

TimeSeriesSplit

ಕಾಲಕ್ರಮದ ದತ್ತಾಂಶಕ್ಕಾಗಿ. **ಭವಿಷ್ಯದ ದತ್ತಾಂಶ ಎಂದಿಗೂ ತರಬೇತಿಗೆ ಬಳಕೆಯಾಗುವುದಿಲ್ಲ**; ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ training set ದೊಡ್ಡದಾಗುತ್ತಾ ಹೋಗುತ್ತದೆ ಮತ್ತು validation ಮುಂದಿನ ಕಾಲ-ಬ್ಲಾಕ್.



ಚಿತ್ರ ೫: 100 ಮಾದರಿಗಳ TimeSeriesSplit (4 ಸುತ್ತು)

ಸುತ್ತು	Training (index)	Validation (index)
0	0–19	20–39
1	0–39	40–59
2	0–59	60–79
3	0–79	80–99

ಹೋಲಿಕೆ: ಮಳೆ ಮುನ್ಸೂಚನೆ

ಜನವರಿ–ಮಾರ್ಚ್ ದತ್ತಾಂಶದಿಂದ ಕಲಿತು ಏಪ್ರಿಲ್ ಅನ್ನು ಊಹಿಸುವುದು ಸರಿ. ಏಪ್ರಿಲ್ ದತ್ತಾಂಶದಿಂದ ಕಲಿತು ಮಾರ್ಚ್ ಅನ್ನು "ಊಹಿಸುವುದು" ಮೋಸ. ಹಣಕಾಸು, ಹವಾಮಾನ, ಉತ್ಪಾದನೆಯಂತಹ ಕಾಲಾಧಾರಿತ ದತ್ತಾಂಶಕ್ಕೆ ಇದೇ ವಿಧಾನ. *ಮಿತಿ:* shuffle ಇಲ್ಲ, stratification ಇಲ್ಲ; ದತ್ತಾಂಶ ಚಿಕ್ಕದಾದರೆ ಕಡಿಮೆ ಪರೀಕ್ಷಾ ಸೆಟ್‌ಗಳು.

ಯಾವುದನ್ನು ಆರಿಸಬೇಕು? (ತ್ವರಿತ ಮಾರ್ಗದರ್ಶಿ)

ದೊಡ್ಡ ದತ್ತಾಂಶ / ಡೀಪ್ ಲರ್ನಿಂಗ್ → **Hold-out** (validation_split). ಸಣ್ಣ ದತ್ತಾಂಶ, classical ML → **K-Fold**. ವರ್ಗಗಳು ಅಸಮತೋಲನ → **Stratified K-Fold**. ಕಾಲ-ಸರಣಿ → **TimeSeriesSplit**. ಅತಿ ಸಣ್ಣ ದತ್ತಾಂಶ → **LOO**.

೧೦. ಪಠ್ಯವನ್ನು ಸಂಖ್ಯೆಗಳಾಗಿಸುವುದು: IMDB ಸೆಂಟಿಮೆಂಟ್ ವಿಶ್ಲೇಷಣೆ

ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಸಂಖ್ಯೆಗಳು ಮಾತ್ರ ಅರ್ಥವಾಗುತ್ತವೆ, ಪದಗಳಲ್ಲ. ಹಾಗಾಗಿ ಪಠ್ಯವನ್ನು ಸಂಖ್ಯೆಗಳಾಗಿ ಬದಲಿಸಬೇಕು. ಇದಕ್ಕೆ ಒಂದು ನೈಜ ಸಮಸ್ಯೆ: **IMDB ಸಿನಿಮಾ ವಿಮರ್ಶೆಗಳ ಸೆಂಟಿಮೆಂಟ್ ವಿಶ್ಲೇಷಣೆ** — ವಿಮರ್ಶೆ ಒಳ್ಳೆಯದೋ (positive) ಕೆಟ್ಟದೋ (negative) ಎಂದು ಊಹಿಸುವುದು.

ದತ್ತಾಂಶ

• 50,000 ವಿಮರ್ಶೆಗಳು (25,000 ತರಬೇತಿ + 25,000 test). • ರೇಟಿಂಗ್ 1–4 → **negative (0)**; ರೇಟಿಂಗ್ 7–10 → **positive (1)**.

ಉದಾಹರಣೆ (ಸ್ಲೈಡ್‌ನಲ್ಲಿ): "*Forrest Gump is one of the best movies of all time... so unforgettable...*" → positive (10/10). "*It's hard to overstate how bad this is... Do not ever watch this!*" → negative (1/10).

ಪಠ್ಯವನ್ನು ಎನ್‌ಕೋಡ್ ಮಾಡುವ ಮೂರು ವಿಧಾನಗಳು

#	ವಿಧಾನ	ಸಾರ
೧	ಸಂಖ್ಯಾ ಎನ್‌ಕೋಡಿಂಗ್ (UTF-8, frequency index)	ಪ್ರತಿ ಪದಕ್ಕೆ ಒಂದು ಪೂರ್ಣಾಂಕ ಸೂಚಿ
೨	One-hot encoding	ಪದಕೋಶದಷ್ಟು ಉದ್ದದ ಸದಿಶ, ಒಂದೇ 1
೩	ಸದಿಶ ನಿರೂಪಣೆ (Embedding layer)	ಪ್ರತಿ ಪದಕ್ಕೆ ಚಿಕ್ಕ, ಕಲಿಯಬಲ್ಲ ಸದಿಶ (word2vec, GloVe ನಂತಹ ಮೊದಲೇ ತರಬೇತಿಯಾದವೂ ಬಳಸಬಹುದು)

Token = ಪದ ಅಥವಾ ಅಕ್ಷರ. ಪ್ರಕ್ರಿಯೆ: **Tokenisation** (ಪಠ್ಯವನ್ನು ತುಂಡುಗಳಾಗಿ ಒಡೆಯುವುದು) → **Vectorisation** → **Encoding**.

Tokenizer (Keras)

ಪ್ಯಾರಾಮೀಟರ್	ಅರ್ಥ
`num_words`	ಪದ ಆವರ್ತನೆಯ ಆಧಾರದ ಮೇಲೆ ಗರಿಷ್ಠ ಎಷ್ಟು ಪದಗಳನ್ನು ಉಳಿಸಬೇಕು (ಉದಾ: 10,000)
`filters`	ತೆಗೆಯಬೇಕಾದ ಚಿಹ್ನೆಗಳು (!"#$%&... ಇತ್ಯಾದಿ)
`lower=True`	ದೊಡ್ಡ-ಚಿಕ್ಕ ಅಕ್ಷರಗಳನ್ನು ಒಂದೇ ಎಂದು ಪರಿಗಣಿಸು
`split=' '`	ಪದಗಳನ್ನು ಖಾಲಿ ಜಾಗದಿಂದ ಬೇರ್ಪಡಿಸು
`char_level=False`	False = ಪದಗಳಾಗಿ ಒಡೆ; True = ಅಕ್ಷರಗಳಾಗಿ ಒಡೆ
`oov_token`	ಪದಕೋಶದಲ್ಲಿ ಇಲ್ಲದ ಪದಗಳ ಬದಲಿ ಚಿಹ್ನೆ (ಉದಾ: '<OOV>')

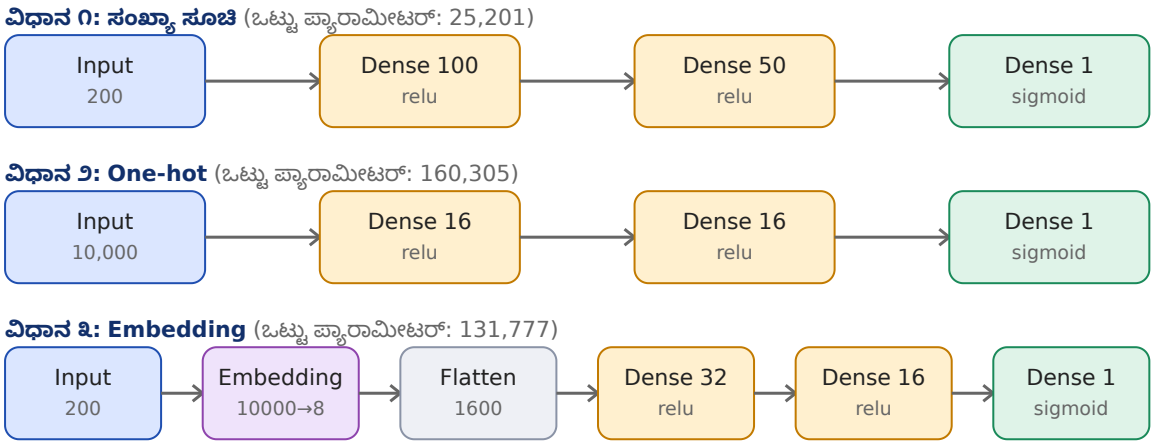
ಸರಳ ಕಲ್ಪಿತ ಉದಾಹರಣೆ: ಟೋಕನೈಸೇಶನ್ ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ

ಪದಕೋಶ (ಆವರ್ತನೆ ಕ್ರಮದಲ್ಲಿ ಸಂಖ್ಯೆ — ನಮ್ಮ ಕಲ್ಪಿತ ಪಟ್ಟಿ): the = 1, movie = 2, great = 3, boring = 4, love = 5.

```
ವಾಕ್ಯ: "I LOVE this Movie!"
1) lower=True          → "i love this movie!"
2) filters ('!' ತೆಗೆ)  → "i love this movie"
3) ಪದಗಳಾಗಿ ಒಡೆ        → ["i", "love", "this", "movie"]
4) ಸಂಖ್ಯೆಗಳಾಗಿ         → ["i"→ಇಲ್ಲ(OOV), love→5, "this"→ಇಲ್ಲ(OOV), movie→2]
                        → [OOV, 5, OOV, 2]
```

ಪದಕೋಶದಲ್ಲಿ ಇಲ್ಲದ ಪದ ಬಂದರೆ ನಿರ್ದಿಷ್ಟ ಬದಲಿ ಚಿಹ್ನೆ (OOV = Out Of Vocabulary) ಬಳಸುತ್ತೇವೆ.

೧೧. ವಿಧಾನ ೧: ಪದ ಸಂಖ್ಯೆಗಳು + Padding + Dense



ಚಿತ್ರ ೬: ಮೂರು ವಿಧಾನಗಳ ನೆಟ್‌ವರ್ಕ್ ವಿನ್ಯಾಸಗಳ ಹೋಲಿಕೆ (ಭಾಗ ೧೧–೧೩)

ಹಂತ ೧: ದತ್ತಾಂಶ ಲೋಡ್

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np, matplotlib.pyplot as plt, tensorflow as tf

max_words = 10000                          # ಅತಿ ಹೆಚ್ಚು ಬಳಕೆಯ 10,000 ಪದಗಳು ಮಾತ್ರ
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_words)
# ವಿಮರ್ಶೆಗಳು ಈಗಾಗಲೇ ಸಂಖ್ಯೆಗಳ ಪಟ್ಟಿಗಳಾಗಿ ಬದಲಾಗಿವೆ; ಪ್ರತಿ ಪದಕ್ಕೆ ಒಂದು ಸೂಚಿ.
# ಪದಗಳ ಸೂಚಿಗಳನ್ನು ಒಟ್ಟಾರೆ ಆವರ್ತನೆಯ ಕ್ರಮದಲ್ಲಿ ನೀಡಲಾಗಿದೆ (ಚಿಕ್ಕ ಸಂಖ್ಯೆ = ಸಾಮಾನ್ಯ ಪದ).
```

ವಿಶೇಷ ಸೂಚಿಗಳು ಮತ್ತು ಪದಕೋಶ

ಸೂಚಿ	ಅರ್ಥ
0	ಖಾಲಿ ಜಾಗ / padding (ಉದ್ದ ಸಮಗೊಳಿಸಲು)
1	ವಿಮರ್ಶೆಯ ಆರಂಭ
2	ಅಜ್ಞಾತ ಪದ (ಅತಿ ಹೆಚ್ಚು ಬಳಕೆಯ 10,000 ರಲ್ಲಿ ಇಲ್ಲದ ಪದ)
3, 4, 5, …	ನಿಜವಾದ ಪದಗಳು

```
word_index = imdb.get_word_index()    # ಪದಕೋಶ: key = ಪದ, value = ಆವರ್ತನೆ ಕ್ರಮದ ಸೂಚಿ
```

ಪ್ರಮುಖ ಅಂಶ: `get_word_index()` ನ ಸೂಚಿಗಳು 1 ರಿಂದ ಆರಂಭ, ಆದರೆ `load_data()` ನ ವಿಮರ್ಶೆಗಳಲ್ಲಿ 0, 1, 2 ವಿಶೇಷ ಬಳಕೆಗೆ ಮೀಸಲಿರುವುದರಿಂದ ಪದ ಸೂಚಿಗಳು **3 ರಿಂದ ಆರಂಭ**. ಅಂದರೆ ಪದಕೋಶದ ಸೂಚಿಗೆ **+3** ಮಾಡಬೇಕು (ಸ್ಲೈಡ್‌ನ “IMDB_extended_index − 3”).

ಹಂತ ೨: ಎಲ್ಲ ವಿಮರ್ಶೆಗಳನ್ನು ಒಂದೇ ಉದ್ದಕ್ಕೆ (200) ತರುವುದು — Padding

ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಸ್ಥಿರ ಗಾತ್ರದ ಇನ್‌ಪುಟ್ ಬೇಕು. ವಿಮರ್ಶೆ ಚಿಕ್ಕದಾದರೆ 0 ಗಳನ್ನು ಸೇರಿಸಿ (padding), ಉದ್ದವಾದರೆ ಹೆಚ್ಚುವರಿ ಪದಗಳನ್ನು ಕತ್ತರಿಸುತ್ತೇವೆ.

```
maxlen = 200
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test  = pad_sequences(x_test,  maxlen=maxlen)
```

ಕಲ್ಪಿತ ಉದಾಹರಣೆ: maxlen = 5

```
ಚಿಕ್ಕ ವಿಮರ್ಶೆ [4, 8, 15]            → [0, 0, 4, 8, 15]        (ಮುಂದೆ 0 ಸೇರ್ಪಡೆ; ಡೀಫಾಲ್ಟ್ padding='pre')
ಉದ್ದ ವಿಮರ್ಶೆ [1, 2, 3, 4, 5, 6, 7]  → [3, 4, 5, 6, 7]         (ಡೀಫಾಲ್ಟ್: ಆರಂಭದಿಂದ ಕತ್ತರಿಸುತ್ತದೆ; ಕೊನೆಯ 5 ಉಳಿಯುತ್ತವೆ)
```

ಹಂತ ೩: ಮಾದರಿ ಮತ್ತು ತರಬೇತಿ

```
model = Sequential()
model.add(Input(shape=(maxlen,)))                # 200 ಸಂಖ್ಯೆಗಳು
model.add(Dense(100, activation='relu'))
model.add(Dense(50,  activation='relu'))         # fully connected ಪದರ
model.add(Dense(1,   activation='sigmoid'))      # 0–1 ನಡುವಿನ ಉತ್ತರ: positive ಸಾಧ್ಯತೆ
model.compile(optimizer='adam',
              loss='binary_crossentropy',        # 2 ವರ್ಗ (0 ಅಥವಾ 1) ಗೆ
              metrics=['accuracy'])
model.summary()

history = model.fit(x_train, y_train, epochs=50, batch_size=32, validation_split=0.2)
scores  = model.evaluate(x_test, y_test, verbose=1)

plt.plot(history.history['loss'],     label='Loss on training set')
plt.plot(history.history['val_loss'], label='Loss on validation set')
plt.xlabel('Training'); plt.ylabel('Loss'); plt.legend(); plt.show()
```

ಲೆಕ್ಕ: ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು ಮತ್ತು ಹೆಜ್ಜೆಗಳು

```
Dense(100): 200 × 100 + 100 = 20,100
Dense(50) : 100 × 50  + 50  =  5,050
Dense(1)  :  50 × 1   + 1   =     51
ಒಟ್ಟು                       = 25,201
ತರಬೇತಿ: 25,000 × 0.8 = 20,000; ಹೆಜ್ಜೆಗಳು/epoch = 20,000 / 32 = 625
50 epochs = 31,250 ತೂಕ ಬದಲಾವಣೆಗಳು
```

> **ಈ ವಿಧಾನದ ದೌರ್ಬಲ್ಯ (ನನ್ನ ವಿಶ್ಲೇಷಣೆ)**
> ಪದ ಸೂಚಿಗಳು ಕೇವಲ ಆವರ್ತನೆ ಕ್ರಮ — ಯಾವುದೇ ಅರ್ಥವಿಲ್ಲ. ಸೂಚಿ 4369 ಇರುವ ಪದ 194 ರ ಪದಕ್ಕಿಂತ "22 ಪಟ್ಟು ಹೆಚ್ಚು" ಎಂದು ನೆಟ್‌ವರ್ಕ್ ತಪ್ಪಾಗಿ ಭಾವಿಸುತ್ತದೆ (ಉದಾ: ಕಲ್ಪಿತವಾಗಿ "great" = 84 ಮತ್ತು "awful" = 300 ಆದರೆ ಈ ಸಂಖ್ಯೆಗಳು ಅರ್ಥದ ಬಗ್ಗೆ ಏನೂ ಹೇಳುವುದಿಲ್ಲ). ಜೊತೆಗೆ 50 epochs ಎಂದರೆ validation loss ಏರಿ overfitting ಆಗುವ ಸಾಧ್ಯತೆ ಹೆಚ್ಚು — ಇಲ್ಲಿ early stopping/dropout ಉಪಯುಕ್ತ. ಇದಕ್ಕೆ ಪರಿಹಾರವೇ ಮುಂದಿನ ಎರಡು ವಿಧಾನಗಳು.

ಹಂತ ೪: ಹೊಸ ವಿಮರ್ಶೆಗೆ ಭವಿಷ್ಯವಾಣಿ (ಸ್ಲೈಡ್‌ನ ಕೋಡ್)

```
tokenizer = Tokenizer(num_words=10000, oov_token='<OOV>',
                      filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n -', lower=True)
tokenizer.word_index = word_index                  # IMDB ಪದಕೋಶವನ್ನು ಟೋಕನೈಸರ್‌ಗೆ ಅನ್ವಯಿಸು

encoded_text = tokenizer.texts_to_sequences([input_text])    # ವಾಕ್ಯ → ಸಂಖ್ಯೆಗಳು
# ಆರಂಭ ಚಿಹ್ನೆ 1 ಸೇರಿಸು; ಪ್ರತಿ ಸೂಚಿಗೆ +3; ಗೊತ್ತಿಲ್ಲದ ಪದ → 2
encoded_text = [[1] + [i + 3 if i is not None else 2 for i in lst] for lst in
encoded_text]
padded = pad_sequences(encoded_text, maxlen=200, padding='pre')   # ಉದ್ದ 200 ಕ್ಕೆ ತರು

prediction = model.predict(padded)                 # 0 ಕ್ಕೆ ಹತ್ತಿರ = negative, 1 ಕ್ಕೆ ಹತ್ತಿರ =
positive
print(prediction)
```

೧೨. ವಿಧಾನ ೨: One-hot (Multi-hot) ಎನ್‌ಕೋಡಿಂಗ್

One-hot encoding (ಇತರ ಹೆಸರುಗಳು: unitary/binary encoding, single-vector encoding, direct encoding) ವರ್ಗೀಯ ಮೌಲ್ಯಗಳನ್ನು ಸಂಖ್ಯೆಗಳಾಗಿ ಬದಲಿಸುವ ವಿಧಾನ. ಪ್ರತಿ ವರ್ಗವನ್ನು **ಒಂದೇ 1 ಇರುವ, ಉಳಿದೆಲ್ಲ 0 ಇರುವ** ಸದಿಶವಾಗಿ ಬರೆಯುತ್ತೇವೆ. ಸದಿಶದ ಉದ್ದ = ವರ್ಗಗಳ (ಇಲ್ಲಿ ಪದಕೋಶದ) ಒಟ್ಟು ಸಂಖ್ಯೆ.

> **ಉದಾಹರಣೆ: ಚಿಕ್ಕ ಪದಕೋಶ (5 ಪದಗಳು)**
>
> ```
> ಪದಕೋಶ: [the, movie, great, boring, love]   ಸೂಚಿಗಳು 0..4
>
> great   → [0, 0, 1, 0, 0]
> boring  → [0, 0, 0, 1, 0]
> ```
>
> ಪದಕೋಶ 10,000 ಇದ್ದರೆ ಪ್ರತಿ ಪದಕ್ಕೆ 10,000 ಉದ್ದದ ಸದಿಶ (ಅದರಲ್ಲಿ 9,999 ಸೊನ್ನೆ) — ತುಂಬ ವ್ಯರ್ಥ.

Python ನ enumerate(): ಪಟ್ಟಿಯ ಜೊತೆ ಎಣಿಕೆ ಸಂಖ್ಯೆಯನ್ನೂ ಕೊಡುತ್ತದೆ — `enumerate(iterable, start=0)` (index, element) ಜೋಡಿಗಳ iterator ಹಿಂದಿರುಗಿಸುತ್ತದೆ. ಉದಾ: `list(enumerate(['a','b']))` = `[(0,'a'), (1,'b')]`.

ವಿಮರ್ಶೆಯನ್ನು ಒಂದೇ ಸದಿಶಕ್ಕೆ — vectorize_sequences

```
import numpy as np

def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))   # (ವಿಮರ್ಶೆಗಳು × 10000) ಸೊನ್ನೆಗಳ ಮ್ಯಾಟ್ರಿಕ್ಸ್
    for i, sequence in enumerate(sequences):          # i = ವಿಮರ್ಶೆ ಸಂಖ್ಯೆ
        results[i, sequence] = 1.                     # ಆ ವಿಮರ್ಶೆಯ ಪ್ರತಿ ಪದದ ಸ್ಥಾನದಲ್ಲಿ 1
    return results
```

ಲೆಕ್ಕ (dimension = 8 ಎಂದು ಸರಳಗೊಳಿಸಿ)

```
ವಿಮರ್ಶೆ A = [1, 3, 4]      → [0, 1, 0, 1, 1, 0, 0, 0]
ವಿಮರ್ಶೆ B = [2, 3, 3, 7]    → [0, 0, 1, 1, 0, 0, 0, 1]   (ಪದ 3 ಎರಡು ಬಾರಿ ಬಂದರೂ ಒಂದೇ 1)
```

ಇದನ್ನು **multi-hot** ಎನ್ನುತ್ತಾರೆ: “ಈ ಪದ ಇದೆಯೇ / ಇಲ್ಲವೇ” ಎಂದು ಮಾತ್ರ ಗುರುತಿಸುತ್ತದೆ; ಪದಗಳ ಕ್ರಮ ಮತ್ತು ಎಷ್ಟು ಬಾರಿ ಬಂದಿತು ಎಂಬುದು ಕಳೆದುಹೋಗುತ್ತದೆ.

```
x_train = vectorize_sequences(x_train)   # 25000 × 10000
x_test  = vectorize_sequences(x_test)

model = models.Sequential()
model.add(Input(shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1,  activation='sigmoid'))
model.compile(optimizer=optimizers.Adam(learning_rate=0.001),
              loss='binary_crossentropy', metrics=['accuracy'])
model.summary()

history = model.fit(x_train, train_labels, epochs=20, batch_size=32,
validation_split=0.4)
scores  = model.evaluate(x_test, test_labels, verbose=1)
prediction = model.predict(xv)           # xv = ಹೊಸ ವಿಮರ್ಶೆಯ ವೆಕ್ಟರೈಸ್ ರೂಪ
```

ಲೆಕ್ಕ: ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು

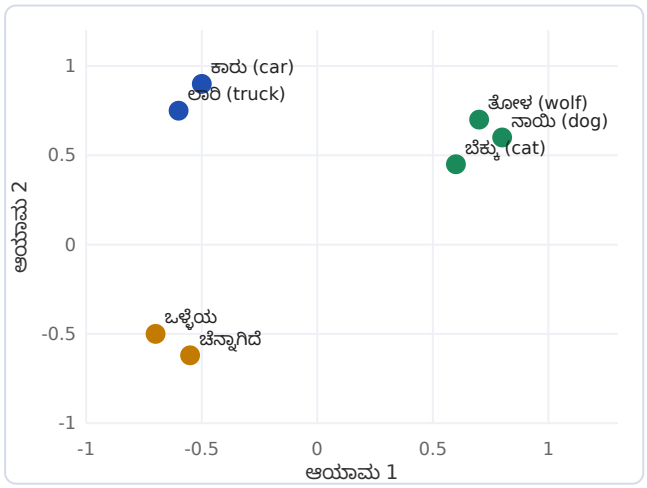
```
Dense(16): 10,000 × 16 + 16 = 160,016
Dense(16):     16 × 16 + 16 =     272
Dense(1) :     16 × 1  + 1  =      17
ಒಟ್ಟು                        = 160,305
validation_split=0.4 → ತರಬೇತಿ 15,000; ಹೆಜ್ಜೆಗಳು/epoch = 15,000/32 = 468.75 → 469
```

ಇನ್‌ಪುಟ್ ದೊಡ್ಡದು (10,000) — ಹಾಗಾಗಿ ಮೊದಲ ಪದರದಲ್ಲೇ ಹೆಚ್ಚಿನ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು (ವಿಧಾನ ೧ ರ 25,201 ಕ್ಕೆ ಹೋಲಿಸಿ ಸುಮಾರು 6 ಪಟ್ಟು). ಇದರಿಂದ overfitting ಅಪಾಯ ಹೆಚ್ಚು — ಈ ಉಪನ್ಯಾಸದ regularisation ತಂತ್ರಗಳು (L2, Dropout) ಇಲ್ಲಿ ಹೆಚ್ಚು ಉಪಯುಕ್ತ (ನನ್ನ ಟಿಪ್ಪಣಿ).

೧೩. ವಿಧಾನ ೩: Word Embeddings

Word embedding ಎಂದರೆ ಪ್ರತಿ ಪದಕ್ಕೆ ಒಂದು **ಚಿಕ್ಕ, ದಟ್ಟ (dense), ನೈಜ ಸಂಖ್ಯೆಗಳ ಸದಿಶ**. ಈ ಸದಿಶಗಳನ್ನು ನೆಟ್‌ವರ್ಕ್ ತರಬೇತಿಯ ಸಮಯದಲ್ಲೇ ತೂಕಗಳೊಂದಿಗೆ ಕಲಿಯುತ್ತದೆ: ಮೊದಲು ಯಾದೃಚ್ಛಿಕ ಸದಿಶಗಳು, ನಂತರ ಹೆಜ್ಜೆ ಹೆಜ್ಜೆಯಾಗಿ ಸುಧಾರಣೆ. ಮೊದಲೇ ತರಬೇತಿಯಾದ ಸದಿಶಗಳನ್ನೂ (word2vec, GloVe) ಲೋಡ್ ಮಾಡಬಹುದು. ದೊಡ್ಡ ಪದಕೋಶಗಳಿಗೆ ಸಾಮಾನ್ಯವಾಗಿ 256, 512 ಅಥವಾ 1024 ಉದ್ದ (ಒಂದೇ-ಬಿಸಿ ಎನ್‌ಕೋಡಿಂಗ್‌ನಲ್ಲಿ 20,000 ಆಗಬಹುದಾದಲ್ಲಿ).

One-hot ಸದಿಶ	Embedding ಸದಿಶ
ವಿರಳ (sparse) — ಬಹುತೇಕ ಸೊನ್ನೆ	ದಟ್ಟ, ಸಾಂದ್ರ (compact-dense)
ಉದ್ದ — ಪದಕೋಶದಷ್ಟು	ಚಿಕ್ಕ
ನಮ್ಯವಲ್ಲದ — ಪದಗಳ ನಡುವೆ ಸಂಬಂಧ ತೋರಿಸುವುದಿಲ್ಲ	ದತ್ತಾಂಶದಿಂದ ಕಲಿತದ್ದು — ಅರ್ಥ ಹಿಡಿಯುತ್ತದೆ



ಚಿತ್ರ ೭: (ಕಲ್ಪಿತ 2-ಆಯಾಮದ ಉದಾಹರಣೆ) ಅರ್ಥದಲ್ಲಿ ಹತ್ತಿರದ ಪದಗಳು ಹತ್ತಿರ ಹತ್ತಿರ ಬೀಳುತ್ತವೆ

ಲೆಕ್ಕ: "ಹತ್ತಿರ" ಎಂದರೇನು? (cosine similarity)

ಕಲ್ಪಿತ ಸದಿಶಗಳು: ನಾಯಿ = (0.8, 0.6), ತೋಳ = (0.7, 0.7), ಕಾರು = (−0.5, 0.9).

```
ನಾಯಿ·ತೋಳ = 0.8×0.7 + 0.6×0.7 = 0.98;   |ನಾಯಿ| = 1.0,   |ತೋಳ| = 0.99
cosine(ನಾಯಿ, ತೋಳ) = 0.98 / (1.0 × 0.99) = 0.99    (ಬಹುತೇಕ ಒಂದೇ ದಿಕ್ಕು → ಅರ್ಥ ಹತ್ತಿರ)

ನಾಯಿ·ಕಾರು = 0.8×(−0.5) + 0.6×0.9 = 0.14;   |ಕಾರು| = 1.03
cosine(ನಾಯಿ, ಕಾರು) = 0.14 / (1.0 × 1.03) = 0.14    (ಬೇರೆ ದಿಕ್ಕು → ಅರ್ಥ ದೂರ)
```

ಇದನ್ನು One-hot ನಲ್ಲಿ ಸಾಧ್ಯವಿಲ್ಲ: ಅಲ್ಲಿ ಯಾವುದೇ ಎರಡು ಬೇರೆ ಪದಗಳ ಸದಿಶಗಳ ಡಾಟ್ ಪ್ರಾಡಕ್ಟ್ ಯಾವಾಗಲೂ 0 — "ನಾಯಿ" ಮತ್ತು "ತೋಳ" ಕೂಡ "ನಾಯಿ" ಮತ್ತು "ಕಾರು" ಎಷ್ಟೇ ದೂರ.

Keras ನ Embedding ಪದರ

ಕನಿಷ್ಠ ಎರಡು ಆರ್ಗ್ಯುಮೆಂಟ್: **ಪದಕೋಶದ ಗಾತ್ರ** (ಸಾಧ್ಯವಿರುವ tokens ಸಂಖ್ಯೆ) ಮತ್ತು **embedding ಆಯಾಮ** (ಸದಿಶದ ಉದ್ದ). ಇದು ಮೂಲತಃ ಒಂದು ಪಟ್ಟಿ (lookup table): ಪದ ಸೂಚಿ → ಅದರ ಸದಿಶ. `Embedding(10000, 64)` ಎಂದರೆ 10,000 ಪದಗಳಿಗೆ ತಲಾ 64 ಉದ್ದದ ಸದಿಶ.

ಸ್ಲೈಡ್‌ನ ಸಣ್ಣ ವ್ಯತ್ಯಾಸ

ಸ್ಲೈಡ್ 39 ರ ಕೋಡ್ `Embedding(1000, 64)` ಎಂದಿದೆ ಆದರೆ ಟಿಪ್ಪಣಿಯಲ್ಲಿ "dictionary size = 10000" ಎಂದಿದೆ — ಇದು ಟೈಪೋ ಇರಬಹುದು. ನಮ್ಮ IMDB ಮಾದರಿಯಲ್ಲಿ 10,000 ಪದ ಇರುವುದರಿಂದ `Embedding(10000, ...)` ಸರಿ.

```
from keras.layers import Embedding

model = models.Sequential()
model.add(Input(shape=(maxlen,)))            # (batch, 200) — ಪದ ಸೂಚಿಗಳು
model.add(Embedding(10000, 8))               # → (batch, 200, 8): ಪ್ರತಿ ಪದಕ್ಕೆ 8 ಉದ್ದದ ಸದಿಶ
model.add(Flatten())                         # → (batch, 200×8 = 1600)
model.add(layers.Dense(32, activation='relu'))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1,  activation='sigmoid'))
# compile / fit / evaluate / predict — ವಿಧಾನ ೨ ರಂತೆಯೇ (binary_crossentropy, adam)
```

ಆಕಾರ ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್ ಲೆಕ್ಕ

```
Input        : (batch, 200)
Embedding    : (batch, 200, 8)     ಪ್ಯಾರಾಮೀಟರ್ = 10,000 × 8      = 80,000
Flatten      : (batch, 1600)                                        0
Dense(32)    : 1,600 × 32 + 32                                    = 51,232
Dense(16)    :    32 × 16 + 16                                    =    528
Dense(1)     :    16 × 1  + 1                                     =     17
ಒಟ್ಟು                                                          = 131,777
```

ವಿಧಾನ ೨ (160,305) ಗಿಂತ ಕಡಿಮೆ ಪ್ಯಾರಾಮೀಟರ್, ಮೇಲಾಗಿ ಪದಗಳ ಅರ್ಥ ಸಂಬಂಧವನ್ನೂ ಕಲಿಯುತ್ತದೆ.

೧೪. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

Overfitting ನಿಯಂತ್ರಣ ತಂತ್ರಗಳ ಸಾರಾಂಶ

ತಂತ್ರ	ಮೂಲ ಕಲ್ಪನೆ	ಮುಖ್ಯ ಸೂಚನೆ
L2 (Ridge)	ತೂಕಗಳ ವರ್ಗ ಮೊತ್ತಕ್ಕೆ ದಂಡ	ತೂಕಗಳನ್ನು ಚಿಕ್ಕದಾಗಿಸುತ್ತದೆ, 0 ಮಾಡುವುದಿಲ್ಲ
L1 (Lasso)	|ತೂಕ| ಮೊತ್ತಕ್ಕೆ ದಂಡ	ಕೆಲವು ತೂಕ ನಿಖರ 0 → feature selection
Elastic Net	L1 + L2 ಮಿಶ್ರಣ	α ಮಿಶ್ರಣ ಪ್ರಮಾಣ
Early stopping	validation ಕೆಟ್ಟಾಗ ನಿಲ್ಲಿಸು	ಸಾಕಷ್ಟು epochs ಕೊಡಿ
Dropout	ಯಾದೃಚ್ಛಿಕ ನ್ಯೂರಾನ್ ಆಫ್, 1/(1 − p) ಸ್ಕೇಲಿಂಗ್	ತರಬೇತಿಯಲ್ಲಿ ಮಾತ್ರ
Batch Norm	mini-batch ನಲ್ಲಿ normalise + γ, β	Dense → BN → Activation

ಪಠ್ಯ ಎನ್‌ಕೋಡಿಂಗ್ ವಿಧಾನಗಳ ಹೋಲಿಕೆ

ವಿಧಾನ	ಇನ್‌ಪುಟ್ ಆಕಾರ	ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು (ಈ ಉಪನ್ಯಾಸದ ಮಾದರಿ)	ಅರ್ಥ ಹಿಡಿಯುತ್ತದೆಯೇ?
೧. ಪದ ಸೂಚಿ + padding	(batch, 200)	25,201	ಇಲ್ಲ (ಸೂಚಿ ಕೇವಲ ಕ್ರಮಾಂಕ)
೨. One-hot / multi-hot	(batch, 10,000)	160,305	ಪದ “ಇದೆ/ಇಲ್ಲ” ಮಾತ್ರ; ಕ್ರಮ ಕಳೆದುಹೋಗುತ್ತದೆ
೩. Embedding	(batch, 200) → (batch, 200, 8)	131,777	ಹೌದು — ಅರ್ಥದಲ್ಲಿ ಹತ್ತಿರದ ಪದಗಳು ಹತ್ತಿರ

ಪದಕೋಶ

ಇಂಗ್ಲಿಷ್	ಕನ್ನಡದಲ್ಲಿ ಸರಳ ಅರ್ಥ
Regularisation	loss ಗೆ ದಂಡ ಸೇರಿಸಿ ಮಾದರಿ ಸರಳವಾಗಿಸುವುದು
L1 / L2 / Elastic Net	$\|w\|$ ದಂಡ / w^2 ದಂಡ / ಎರಡರ ಮಿಶ್ರಣ
λ (lambda)	ದಂಡದ ಶಕ್ತಿ
Sparse model	ಹೆಚ್ಚು ತೂಕ 0 ಇರುವ ವಿರಳ ಮಾದರಿ
Dropout	ತರಬೇತಿಯಲ್ಲಿ ಯಾದೃಚ್ಛಿಕ ನ್ಯೂರಾನ್‌ಗಳನ್ನು ಆಫ್ ಮಾಡುವುದು
Batch Normalization	mini-batch ನ ಮೌಲ್ಯಗಳನ್ನು ಸರಾಸರಿ 0, ವ್ಯತ್ಯಾಸ 1 ಮಾಡುವುದು
Cross-validation	ದತ್ತಾಂಶವನ್ನು ಹಲವು ಬಾರಿ ಬೇರೆ ಬೇರೆ ರೀತಿ ವಿಭಜಿಸಿ ಮೌಲ್ಯಮಾಪನ ಮಾಡುವುದು
Fold	K-fold ನ ಒಂದು ಭಾಗ
Stratified	ವರ್ಗ ಅನುಪಾತ ಉಳಿಸುವ
Data leakage	test/ಭವಿಷ್ಯದ ಮಾಹಿತಿ ತರಬೇತಿಗೆ ಸೋರುವುದು
Tokenisation	ಪಠ್ಯವನ್ನು ಪದ/ಅಕ್ಷರಗಳಾಗಿ ಒಡೆಯುವುದು
OOV (out of vocabulary)	ಪದಕೋಶದಲ್ಲಿ ಇಲ್ಲದ ಪದ
Padding	ಉದ್ದ ಸಮಗೊಳಿಸಲು 0 ಸೇರಿಸುವುದು
One-hot / multi-hot	ಒಂದೇ 1 / ಹಲವು 1 ಇರುವ 0-1 ಸದಿಶ
Embedding	ಪದಕ್ಕೆ ಕಲಿಯಬಲ್ಲ ಚಿಕ್ಕ ದಟ್ಟ ಸದಿಶ
Sentiment analysis	ಪಠ್ಯದ ಭಾವನೆ (ಒಳ್ಳೆಯ/ಕೆಟ್ಟ) ಗುರುತಿಸುವುದು

ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

1. $w = [2, -1, 0.5]$, $\lambda = 0.1$ ಆದರೆ L1 ಮತ್ತು L2 ದಂಡ ಎಷ್ಟು?
2. ಅಪ್ರಸ್ತುತ features ಹೆಚ್ಚಿರುವ ದತ್ತಾಂಶಕ್ಕೆ L1 ಅಥವಾ L2 — ಯಾವುದು ಒಳ್ಳೆಯದು, ಏಕೆ?
3. Dropout rate = 0.5 ಆದರೆ ಸಕ್ರಿಯ ನ್ಯೂರಾನ್‌ಗಳ ಮೌಲ್ಯವನ್ನು ಎಷ್ಟರಿಂದ ಗುಣಿಸಬೇಕು? ಉಳಿದ ಮೊತ್ತ $S_q = 1.5$ ಆದರೆ ಸ್ಕೇಲ್ ಮಾಡಿದ ಮೊತ್ತ ಎಷ್ಟು?
4. Batch Norm: mini-batch [1, 3, 5, 7] ನ μ ಮತ್ತು σ^2 ಎಷ್ಟು?
5. 1000 ಮಾದರಿಗಳ ದತ್ತಾಂಶಕ್ಕೆ 5-fold CV ಯಲ್ಲಿ ಪ್ರತಿ fold ನ validation ಗಾತ್ರ ಮತ್ತು training ಗಾತ್ರ ಎಷ್ಟು? ಒಟ್ಟು ಎಷ್ಟು ಬಾರಿ ತರಬೇತಿ?
6. ಷೇರು ಬೆಲೆ ಮುನ್ಸೂಚನೆಗೆ ಯಾವ validation ವಿಧಾನ ಬಳಸಬೇಕು ಮತ್ತು ಏಕೆ?
7. maxlen = 4 ಆದಾಗ [7, 9] ಮತ್ತು [1, 2, 3, 4, 5, 6] ವಿಮರ್ಶೆಗಳು pad_sequences ನಂತರ ಹೇಗಿರುತ್ತವೆ (ಡೀಫಾಲ್ಟ್)?
8. Embedding(5000, 16) ನಲ್ಲಿ ಎಷ್ಟು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು? Input ಉದ್ದ 100 ಆದರೆ Embedding ಔಟ್‌ಪುಟ್ ಆಕಾರ ಮತ್ತು Flatten ನಂತರದ ಆಕಾರ?

ಉತ್ತರಗಳು: (೧) L1 = 0.1×(2+1+0.5) = **0.35**; L2 = 0.1×(4+1+0.25) = **0.525**. (೨) **L1** — ಅಪ್ರಸ್ತುತ features ನ ತೂಕ ನಿಖರ 0 ಮಾಡುತ್ತದೆ (feature selection). (೩) $1/(1 - 0.5) = $ **2**; $1.5 \times 2 = $ **3.0**. (೪) $\mu = 16/4 = $ **4**; $\sigma^2 = (9 + 1 + 1 + 9)/4 = $ **5**. (೫) validation = 200, training = 800; ಒಟ್ಟು **5 ಬಾರಿ** ತರಬೇತಿ. (೬) **TimeSeriesSplit** — ಭವಿಷ್ಯದ ದತ್ತಾಂಶ ತರಬೇತಿಗೆ ಸೋರಬಾರದು (data leakage). (೭) [7, 9] → **[0, 0, 7, 9]**; [1,2,3,4,5,6] → **[3, 4, 5, 6]** (ಆರಂಭದಿಂದ ಕತ್ತರಿಸುತ್ತದೆ). (೮) 5000 × 16 = **80,000**; Embedding ಔಟ್‌ಪುಟ್ (batch, 100, 16); Flatten ನಂತರ (batch, **1600**).